part02_architecture — Google ADK Architecture & Event Lifecycle

> Paired with Lecture Notes **Part 2 · 02. Google ADK Architecture** (Slides 16–29).

This lab suite provides hands-on practice to understand and verify Google ADK's (Agent Development Kit) core execution architecture—including **Runner**, **Agent**, **Event Stream**, and **Dual-Layer Memory (Short-term Session vs. Long-term Memory)**—across multiple execution patterns (Standard ADK CLI/Web, Direct Runner, and FastAPI web services).

---

1. Directory Structure & Submodule Summary

```
part02_architecture/
├── adk-based_Agent/                # Collection of standard ADK agents (adk web / adk run)
│   ├── chat_time_weather_agent/    # 🌦️ Worldwide time & weather lookup and general conversation agent
│   ├── memory_layers_agent/        # 🧠 Dual-layer memory (Session State + Memory Service) agent
│   └── README-EN.md                # Integrated guide for adk-based_Agent
├── Direct_ADK/                     # ⚙️ Direct Runner control and extension practice scripts
│   ├── _path.py                    # Common package (common) and path initialization helper
│   ├── direct-adk-simple-*.py      # Basic execution: Sync vs. Async comparison
│   ├── run_events.py               # 🔧 Observe full Event lifecycle of a single request
│   ├── run_workflow_events.py      # 🟢 LLM-less workflow node routing and event observation
│   ├── run_memory_layers.py        # 🟢/🔧 Session (short-term) vs. Memory (long-term) comparison
│   ├── direct-agent-ref-session.py  # InMemoryMemoryService multi-session memory recall
│   ├── direct-adk-working_memory*.py# Working memory and conversational chatbot using ToolContext.state
│   ├── direct-adk-agent_mod*.py    # Real-time Open-Meteo integration & LLM context visualizer (Plugin)
│   ├── direct-adk-fastapi-*.py     # Agent REST API server (FastAPI) and CLI client extension
│   ├── Session_management/         # Session state rollback practice based on Runner.rewind_async()
│   └── README-EN.md                # Detailed guide for Direct_ADK 15 scripts
└── README-EN.md                    # This document (Integrated summary guide)
```

---

2. Core Hands-on Content by Subfolder

① `adk-based_Agent/` — Standard ADK Agent Packages

Modular agents adhering to Google ADK's standard interfaces (`adk run` , `adk web`) and `AgentLoader` specifications.

- `chat_time_weather_agent` (Detailed Guide):
  - **Core Features**: Urban geocoding via Open-Meteo API (`geocode`), local time based on IANA timezones (`get_current_time`), and real-time weather using WMO weather interpretation codes (`get_weather`).
  - **Intelligent Routing**: Automatically invokes tools for time/weather queries, while routing general knowledge, coding, or chit-chat queries directly to the LLM. Includes an offline fallback DB.
- `memory_layers_agent` (Detailed Guide):
  - **State Prefix Scopes**: `user:<key>` (persisted across sessions for a given user), `key` (single-session scope), global `app:<key>` , and ephemeral `temp:<key>` .
  - **Instruction Template Substitution**: Injects short-term state into prompts automatically via `{user:name?}` , `{trip?}` , etc.
  - **Long-term Memory Integration**: Employs `preload_memory` (automatic pre-retrieval injected per turn) and `load_memory` (agent active search tool).

② `Direct_ADK/` — Direct Code-Level Runner Control & Extensions

Assembles and controls `Runner` , `SessionService` , and `MemoryService` directly in Python code without the ADK CLI (Detailed Guide).

1. `direct-adk-simple-async.py` ,.
    - Compares the structure and blocking/non-blocking I/O handling of `runner.run()` (sync iterator) vs. `runner.run_async()` (async generator).
2. `direct-agent-reference.py` ,.
    - `① Tool call requested → ② Runner executes tool → ③ Final synthesized response` .
    - Observes ADK 2.0 Workflow node routing (`route`) and metadata propagation (`node_info` , `output`).
3. **Session & Memory Layer Control (`run_memory_layers.py` , `direct-agent-ref-session.py` , `Session_management/`)**:
    - Compares short-term session State and long-term Memory across storage scopes, search mechanisms (exact match vs. semantic vector search), and lifecycles.
    - Implements session state rollback (Undo) to a specific previous invocation using `Runner.rewind_async()` .
    - Implements a conversational travel booking chatbot utilizing `ToolContext.state` for working memory accumulation.
4. `direct-adk-fastapi-*.py` ,.
    - Leverages `BasePlugin` to inspect the exact Context Window (system instructions, tool schemas, conversation history) sent to the LLM along with token usage metrics in real time.
    - Wraps agents inside a FastAPI asynchronous REST API server and communicates with an interactive terminal client.

---

3. Quick Command Reference

① Standard Interfaces (`adk-based_Agent`)

```
cd part02_architecture/adk-based_Agent

# Launch Web UI (http://localhost:8008)
adk web --port 8008

# Interactive CLI execution
adk run chat_time_weather_agent
adk run memory_layers_agent
```

② Core Observation Scripts (`Direct_ADK`)

```
cd part02_architecture/Direct_ADK

# 1. Observe Event lifecycle (🔑 Requires Gemini API key)
uv run run_events.py

# 2. Observe Workflow node routing events (🟢 No API key required)
uv run run_workflow_events.py

# 3. Dual-layer memory (Session vs. Memory) comparison (🟢 Offline / 🔑 LLM)
uv run run_memory_layers.py offline
uv run run_memory_layers.py llm

# 4. Monitor full LLM Context Window
uv run direct-adk-agent_mod-context.py

# 5. Start FastAPI agent server
uv run direct-adk-fastapi-server.py
```